

```

public int[] sortByParityII(int[] nums) {
    int n = nums.length;
    for (int i = 0; i < nums.length; i++) {
        if (i % 2 == 0 && nums[i] % 2 != 0) { // even idx, odd val
            for (int j = i + 1; j < n; j++) {
                if (nums[j] % 2 == 0) { // even val
                    int temp = nums[j];
                    nums[j] = nums[i];
                    nums[i] = temp;
                    break;
                }
            }
        } else if (i % 2 != 0 && nums[i] % 2 == 0) { // odd idx, even val
            for (int j = i + 1; j < n; j++) {
                if (nums[j] % 2 != 0) { // odd val
                    int temp = nums[j];
                    nums[j] = nums[i];
                    nums[i] = temp;
                    break;
                }
            }
        }
    }
    return nums;
}

```

## Selection sort

|   |   |
|---|---|
| i | j |
| 7 | 4 |
| 2 | 3 |
| 1 | 9 |

minIdx = i => 7

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 7 | 4 | 2 | 3 | 1 | 9 |
|---|---|---|---|---|---|

minIdx = j => 4

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 7 | 4 | 2 | 3 | 1 | 9 |
|---|---|---|---|---|---|

minIdx = j => 2

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 7 | 4 | 2 | 3 | 1 | 9 |
|---|---|---|---|---|---|

minIdx = j => 2

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 7 | 4 | 2 | 3 | 1 | 9 |
|---|---|---|---|---|---|

minIdx = j => 1

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 7 | 4 | 2 | 3 | 1 | 9 |
|---|---|---|---|---|---|

minIdx = j => 1

Swap :-

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 1 | 4 | 2 | 3 | 7 | 9 |
|---|---|---|---|---|---|

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 1 | 2 | 4 | 3 | 7 | 9 |
|---|---|---|---|---|---|

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 7 | 9 |
|---|---|---|---|---|---|

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 7 | 9 |
|---|---|---|---|---|---|

```

public static void SelectionSort(int arr[]){
    int n = arr.length;
    for(int i=0;i<n-1;i++){
        int minIdx = i;
        for(int j=i+1;j<n;j++){
            if(arr[j]<arr[minIdx]){
                minIdx = j;
            }
        }
        int temp = arr[i];
        arr[i] = arr[minIdx];
        arr[minIdx] = temp;
    }
    for(int i=0;i<n;i++){
        System.out.print(arr[i]+" ");
    }
}

```

### Insertion Sort.

| j | i |
|---|---|
| 7 | 4 |
| 2 | 3 |
| 1 | 9 |

key = arr[i] = 4.

| j |
|---|
| 7 |
| 4 |
| 2 |
| 3 |
| 1 |
| 9 |

while (j >= 0 && arr[j] > key) {

arr[j+1] = arr[j];

j

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 7 | 7 | 2 | 3 | 1 | 9 |
|---|---|---|---|---|---|

j--;

arr[j+1] = key.

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 4 | 7 | 2 | 3 | 1 | 9 |
|---|---|---|---|---|---|

| j |
|---|
| 7 |
| 4 |
| 4 |
| 2 |
| 3 |
| 9 |

while (j >= 0 && arr[j] > key)

arr[j+1] = arr[j];

j

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 7 | 7 | 4 | 2 | 3 | 9 |
|---|---|---|---|---|---|

arr[j+1] = key.

i → should be starts from 1st position.

j → should be starts from i-1 position.

j = i-1;

j i

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 7 | 4 | 2 | 3 | 1 | 9 |
|---|---|---|---|---|---|

key = arr[i] = 1.

while (j >= 0 && arr[j] > key)

j i

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 7 | 4 | 2 | 3 | 1 | 9 |
|---|---|---|---|---|---|

arr[j+1] = arr[j];

j i

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 7 | 4 | 2 | 3 | 3 | 9 |
|---|---|---|---|---|---|

j--;

j i

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 7 | 4 | 2 | 3 | 3 | 9 |
|---|---|---|---|---|---|

while (j >= 0 && arr[j] > key)

arr[j+1] = arr[j];

j

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 7 | 4 | 2 | 2 | 3 | 9 |
|---|---|---|---|---|---|

j--;

```

public static void InsertionSort(int arr[]){
    int n = arr.length;
    for(int i=1;i<n;i++){
        int j = i-1;
        int key = arr[i];
        while(j>=0 && arr[j]>key){
            arr[j+1] = arr[j];
            j--;
        }
        arr[j+1] = key;
    }
    for(int i=0;i<n;i++){
        System.out.print(arr[i]+" ");
    }
}

```

## 75. Sort Colors

**Input:** nums = [2,0,2,1,1,0]

**Output:** [0,0,1,1,2,2]

```

public void sortColors(int[] arr) {
    int n = arr.length;
    for(int i=0;i<n-1;i++){
        int minIdx = i;
        for(int j=i+1;j<n;j++){
            if(arr[j]<arr[minIdx]){
                minIdx = j;
            }
        }
        int temp = arr[i];
        arr[i] = arr[minIdx];
        arr[minIdx] = temp;
    }
}

```

## 274. H-Index

**Input:** citations = [3,0,6,1,5]

**Output:** 3

**Explanation:** [3,0,6,1,5] means the researcher has 5 papers in total and each of them had received 3, 0, 6, 1, 5 citations respectively.

Since the researcher has 3 papers with at least 3 citations each and the remaining two with no more than 3 citations each, their h-index is 3.

```

public int hIndex(int[] arr) {
    int n = arr.length;
    for (int i = 0; i < n - 1; i++) {
        int maxIdx = i;
        for (int j = i + 1; j < n; j++)
            if (arr[j] > arr[maxIdx]) {
                maxIdx = j;
            }
        int temp = arr[i];
        arr[i] = arr[maxIdx];
        arr[maxIdx] = temp;
    }
    int h = 0;
    for (int i = 0; i < n; i++) {
        if (arr[i] >= i + 1) {
            h++;
        }
    }
    return h;
}

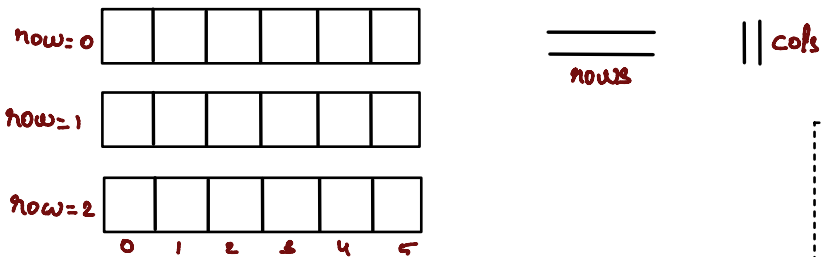
```

## 2D Array.

2D Array is like matrix format.

It contains rows & cols.

Collection of 1D array is known as 2D array.



row length = arr.length  
col length = arr[0].length

for (i=0; i<n; i++) → row  
for (j=0; j<n; j++) → col.

Syntax:  $\text{Datatype ArrayName}[\overset{\text{row}}{r}][\underset{\text{col}}{c}] = \text{new Datatype}[\text{rowSize}][\text{colSize}];$

```
public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    int rowSize = sc.nextInt();
    int colSize = sc.nextInt();
    int arr[][] = new int[rowSize][colSize];
    for(int i=0; i<rowSize; i++){
        for(int j=0; j<colSize; j++){
            arr[i][j] = sc.nextInt();
        }
    }
    for(int i=0; i<rowSize; i++){
        for(int j=0; j<colSize; j++){
            System.out.print(arr[i][j] + " ");
        }
        System.out.println();
    }
}
```

op:-

```
3
3
1 2 3 4 5 6 7 8 9
1 2 3
4 5 6
7 8 9
```

```
public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    int rowSize = sc.nextInt();
    int colSize = sc.nextInt();
    int arr[][] = new int[rowSize][colSize];
    for(int i=0; i<rowSize; i++){
        for(int j=0; j<colSize; j++){
            arr[i][j] = sc.nextInt();
        }
    }
    for(int i=0; i<rowSize; i++){
        for(int j=colSize-1; j>=0; j--){
            System.out.print(arr[i][j] + " ");
        }
        System.out.println();
    }
}
```

```
3
3
1 2 3 4 5 6 7 8 9
3 2 1
6 5 4
9 8 7
```

## Wave Traversal 1

|    |    |    |    |    |
|----|----|----|----|----|
| 1  | 2  | 3  | 4  | 5  |
| 6  | 7  | 8  | 9  | 10 |
| 15 | 14 | 13 | 12 | 11 |
| 16 | 17 | 18 | 19 | 20 |
| 25 | 24 | 23 | 22 | 21 |

|   |    |    |    |    |    |
|---|----|----|----|----|----|
|   | c  | c  | c  | c  |    |
| n | 1  | 2  | 3  | 4  | 5  |
| n | 6  | 7  | 8  | 9  | 10 |
| n | 15 | 14 | 13 | 12 | 11 |
| n | 16 | 17 | 18 | 19 | 20 |
| n | 25 | 24 | 23 | 22 | 21 |



|    |    |    |    |    |
|----|----|----|----|----|
| 1  | 6  | 15 | 16 | 25 |
| 24 | 17 | 14 | 7  | 2  |
| 3  | 8  | 13 | 18 | 23 |
| 22 | 19 | 12 | 9  | 4  |
| 5  | 10 | 11 | 20 | 21 |

|    |    |    |    |    |
|----|----|----|----|----|
| i  | j  |    |    |    |
| 00 | 10 | 20 | 30 | 40 |
| 41 | 31 | 21 | 11 | 01 |
| 02 | 12 | 22 | 32 | 42 |
| 43 | 33 | 23 | 13 | 03 |
| 04 | 14 | 24 | 34 | 44 |

n → update .

c → same .

col odd index ⇒ reverse .

```
public static void WaveTraversal(int arr[][]){
    int r = arr.length;
    int c = arr[0].length;
    for(int j=0;j<c;j++){
        if(j%2==0){
            for(int i=0;i<r;i++){
                System.out.print(arr[i][j]+" ");
            }
            System.out.println();
        }else{
            for(int i=r-1;i>=0;i--){
                System.out.print(arr[i][j]+" ");
            }
            System.out.println();
        }
    }
}
```

ip: 4 4  
 1 2 3 4  
 5 6 7 8  
 9 10 11 12  
 13 14 15 16  
 -----  
 op: 1 5 9 13  
 14 10 6 2  
 3 7 11 15  
 16 12 8 4

Note:

\* If j idx remains the same for 1st row.  
 write j as outer loop & i as inner loop.

\* If i idx remains the same for 1st row.  
 write i as outer loop & j as inner loop.

## Wave Traversal 2.

|    |    |    |    |    |
|----|----|----|----|----|
| 1  | 2  | 3  | 4  | 5  |
| 6  | 7  | 8  | 9  | 10 |
| 15 | 14 | 13 | 12 | 11 |
| 16 | 17 | 18 | 19 | 20 |
| 25 | 24 | 23 | 22 | 21 |

|    |    |    |    |    |
|----|----|----|----|----|
| 1  | 2  | 3  | 4  | 5  |
| 6  | 7  | 8  | 9  | 10 |
| 15 | 14 | 13 | 12 | 11 |
| 16 | 17 | 18 | 19 | 20 |
| 25 | 24 | 23 | 22 | 21 |

```

public static void WaveTraversal2(int arr[][]) {
    int r = arr.length;
    int c = arr[0].length;
    for(int j=0; j<c; j++) {
        if(j%2!=0) {
            for(int i=0; i<r; i++) {
                System.out.print(arr[i][j]+" ");
            }
            System.out.println();
        } else {
            for(int i=r-1; i>=0; i--) {
                System.out.print(arr[i][j]+" ");
            }
            System.out.println();
        }
    }
}

```

### Wave Traversal 3 .

|    |    |    |    |    |
|----|----|----|----|----|
| 1  | 2  | 3  | 4  | 5  |
| 6  | 7  | 8  | 9  | 10 |
| 15 | 14 | 13 | 12 | 11 |
| 16 | 17 | 18 | 19 | 20 |
| 25 | 24 | 23 | 22 | 21 |

|    |    |    |    |    |
|----|----|----|----|----|
| 1  | 2  | 3  | 4  | 5  |
| 6  | 7  | 8  | 9  | 10 |
| 15 | 14 | 13 | 12 | 11 |
| 16 | 17 | 18 | 19 | 20 |
| 25 | 24 | 23 | 22 | 21 |

```

public static void WaveTraversal3(int arr[][]) {
    int r = arr.length;
    int c = arr[0].length;
    for(int i=0; i<r; i++) {
        if(i%2==0) {
            for(int j=0; j<c; j++) {
                System.out.print(arr[i][j]+" ");
            }
            System.out.println();
        } else {
            for(int j=c-1; j>=0; j--) {
                System.out.print(arr[i][j]+" ");
            }
            System.out.println();
        }
    }
}

```

### Wave Traversal 4 .

|    |    |    |    |    |
|----|----|----|----|----|
| 1  | 2  | 3  | 4  | 5  |
| 6  | 7  | 8  | 9  | 10 |
| 15 | 14 | 13 | 12 | 11 |
| 16 | 17 | 18 | 19 | 20 |
| 25 | 24 | 23 | 22 | 21 |

|    |    |    |    |    |
|----|----|----|----|----|
| 1  | 2  | 3  | 4  | 5  |
| 6  | 7  | 8  | 9  | 10 |
| 15 | 14 | 13 | 12 | 11 |
| 16 | 17 | 18 | 19 | 20 |
| 25 | 24 | 23 | 22 | 21 |

```

public static void WaveTraversal4(int arr[][]) {
    int r = arr.length;
    int c = arr[0].length;
    for(int i=0; i<r; i++) {
        if(i%2!=0) {
            for(int j=0; j<c; j++) {
                System.out.print(arr[i][j]+" ");
            }
            System.out.println();
        } else {
            for(int j=c-1; j>=0; j--) {
                System.out.print(arr[i][j]+" ");
            }
            System.out.println();
        }
    }
}

```

### Sum of upper and lower triangles

Input: n = 3, mat[][] = [[6, 5, 4],  
[1, 2, 5],  
[7, 9, 7]]

Output: [29, 32]

|    |    |    |
|----|----|----|
| 00 | 01 | 02 |
| 10 | 11 | 12 |
| 20 | 21 | 22 |

upper triangle

i=0 j=0      i<=j  
i=0 j=1      i<=j

|    |    |    |
|----|----|----|
| 00 | 01 | 02 |
| 10 | 11 | 12 |
| 20 | 21 | 22 |

Lower triangle.

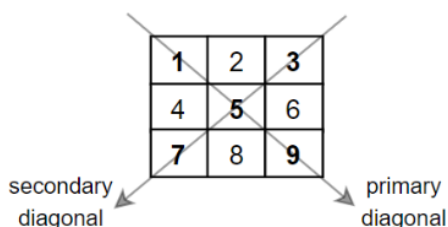
i=1 j=0      i>=j.

```

public ArrayList<Integer> sumTriangles(int mat[][]) {
    ArrayList<Integer> al = new ArrayList<>();
    int upper = 0;
    int lower = 0;
    for(int i=0; i<mat.length; i++){
        for(int j=0; j<mat[0].length; j++){
            if(i<=j){
                upper += mat[i][j];
            } if(i>=j){
                lower += mat[i][j];
            }
        }
    }
    al.add(upper);
    al.add(lower);
    return al;
}

```

### 1572. Matrix Diagonal Sum



Input: mat = [[1,2,3],  
[4,5,6],  
[7,8,9]]

Output: 25

Explanation: Diagonals sum: 1 + 5 + 9 + 3 + 7 = 25  
Notice that element mat[1][1] = 5 is counted only once.

### Approach 1:

```
public int diagonalSum(int[][] mat) {
    int sum = 0;
    for(int i=0;i<mat.length;i++){
        for(int j=0;j<mat[0].length;j++){
            if(i==j || i+j == mat.length-1){
                sum += mat[i][j];
            }
        }
    }
    return sum;
}
```

### Approach 2:

```
public int diagonalSum(int[][] mat) {
    int sum = 0;
    int i = 0;
    int j = mat.length - 1;
    for (int k = 0; k < mat.length; k++) {
        if (i != j) {
            sum += mat[k][i] + mat[k][j];
        } else {
            sum += mat[k][i];
        }
        i++;
        j--;
    }
    return sum;
}
```

---